

LogicaMate

Manual Técnico

1. Stack

Kotlin 2.0.21 · Jetpack Compose (BOM 2024.09.00) + Material 3 · Navigation Compose 2.8.0 · Room 2.6.1 + KSP · Coroutines/Flow · MVVM + Repository · DI manual (sin Hilt) · minSdk 24 / compileSdk-targetSdk 34 · JDK 17 · Gradle Wrapper 8.7 · AGP 8.5.2.

Sin permisos Android. Sin Firebase/backend/APIs remotas/login/ads/analytics.

2. Arquitectura

```
app/src/main/java/com/educalab/logicamate/
■■■ domain/
■ ■■■ model/ Challenge, PieceSpec, enums (ChamberId, LogicCategory, DifficultyLevel...)
■ ■■■ engine/ 11 motores puros (sin Android, 100% testeables en JVM)
■■■ data/
■ ■■■ local/ Room: entidades, DAOs, LogicaMateDatabase, DatabaseSeeder, ChallengeMapper
■ ■■■ repository/ ProgressRepository (cascada intento -> XP -> racha -> progreso ->
fragmento -> insignia)
■■■ seed/ SeedContent.kt (135 desafíos, generado por tools/generate_seed_content.py)
■■■ ui/
■ ■■■ theme/ Color.kt, Type.kt, Theme.kt
■ ■■■ common/ PieceView, ChamberChrome, ChamberViewModel, ChamberScreenScaffold,
PieceCompletionScreen
■ ■■■ navigation/ LogicaMateNavHost (rutas: map, profile, collection, daily, chamber/{ID})
■ ■■■ screens/ 10 paquetes, uno por cámara + home/profile/collection/daily
■■■ MainActivity.kt
■■■ LogicaMateApp.kt (Application: dispara DatabaseSeeder.seedIfNeeded() en IO al arrancar)
■■■ ServiceLocator.kt (DI manual: instancias singleton de Database/Repository)
```

Separación estricta domain/ / data/ / ui/ (sección 22 del prompt maestro). Ninguna regla de negocio vive dentro de un @Composable; toda regla vive en domain/engine y se invoca desde ChamberViewModel/ChallengeValidator.

3. Los 11 motores (domain/engine)

SequenceEngine	Detecta y valida secuencias aritméticas, geométricas y de alternancia de dos pasos
PatternEngine	Detecta ciclos de una propiedad y de propiedades independientes combinadas
ClassificationEngine	Calcula la partición canónica de un conjunto de piezas por una propiedad; compara agrupaciones sin importar el nombre/orden del grupo

Nombre	Responsabilidades
MatrixEngine	Construye/resuelve matrices Raven-style vía una transformación consistente (CellTransform) aplicada fila a fila
AnalogyEngine	Reutiliza MatrixEngine.CellTransform para validar A:A'::B:?
RelationEngine	Búsqueda exhaustiva de permutaciones válidas bajo restricciones de orden; garantiza solución única
DeductionEngine	Fuerza bruta sobre biyecciones persona→objeto bajo pistas; garantiza solución única
ConstructorEngine	Valida una tira libremente construida contra un objetivo (ConstructionGoal)
GamificationEngine	XP, nivel (umbral triangular), racha (por día, sin java.time), reglas de desbloqueo de insignias
ProgressEngine	Deriva el estado visual de cada cámara (bloqueada/disponible/iniciada/completada/dominada) y el desbloqueo de la Cámara Maestra
HintEngine	Política de revelado progresivo de pistas (no se puede saltar niveles)
ChallengeValidator	Punto único de validación de respuestas; despacha según la categoría a la representación correcta (piezas, particiones, órdenes, asignaciones, o el ConstructorEngine)
DailyChallengeGenerator	Generación determinista (semilla = hash de la fecha) del Reto Diario, validando unicidad de solución antes de devolver el desafío

4. Persistencia (Room)

18 entidades (ver docs/BASE_DE_DATOS.md y database/schema.sql para el esquema completo). DatabaseSeeder puebla cámaras, categorías, insignias, fragmentos, coleccionables, el perfil por defecto y — solo si la tabla challenge está vacía — los 135 desafíos semilla desde SeedContent.kt. Es idempotente: puede llamarse en cada arranque sin duplicar datos (verificado en DatabaseSeederTest).

ProgressRepository.recordAttempt(...) es el único punto de escritura para el resultado de un intento: persiste el intento, calcula XP (GamificationEngine), actualiza racha, recalcula el estado de la cámara (ProgressEngine), desbloquea el fragmento de la Llave si corresponde, y evalúa nuevas insignias — todo en una sola llamada, verificado de extremo a extremo en ProgressRepositoryTest (Room en memoria vía Robolectric).

5. Contenido semilla: generación y garantía de corrección

`tools/generate_seed_content.py` genera los 135 desafíos. Es determinista (semilla fija) y **auto-verifica cada desafío antes de emitirlo**: reimplementa en Python, término a término, la misma lógica que los motores Kotlin (`mirror_next_in_cycle`, `mirror_sequence_rule`, `mirror_transform_apply`, `mirror_canonical_partition`, `mirror_relation_unique`, `mirror_deduction_unique`) y usa `assert` sobre cada desafío generado. El script se ejecutó realmente en este entorno (no es una promesa sin comprobar) y terminó sin errores, con la distribución exacta documentada en `MEMORIA_DESCRIPTIVA.md`.

Esto no sustituye ejecutar los tests JVM reales (el entorno de generación no tiene JDK+Kotlin, ver `BUILD_REPORT.md`), pero `SeedContentConsistencyTest.kt` (13 tests) vuelve a comprobar, esta vez con los motores Kotlin reales, que cada desafío semilla es resuelto correctamente — es la prueba que correría en CI y que da la garantía final.

6. UI / Compose

- **Vocabulario visual único**: `PieceSpec` (forma + color + tamaño + cantidad + valor opcional) se dibuja siempre con `PieceView` sobre un Canvas real (triángulo, círculo, cuadrado, estrella, hexágono, diamante) — nunca emojis ni iconos genéricos de Material.
- Un `ChamberViewModel` **compartido** gestiona la cola de desafíos de cualquier cámara: carga, pistas reveladas, envío de respuesta vía `ChallengeValidator`, registro del intento vía `ProgressRepository`, y avance al siguiente desafío.
- **Mecánicas por cámara** (sección 21 del prompt maestro): completar-hueco por toque (Patrones/Secuencias/Analogías/Matrices, con `CompletionLayout` distinto por cámara), clasificar por toque-y-portal (Clasificación), ordenar por toque secuencial (Relaciones), asignar en rejilla (Deducción), construir libremente (Constructor).
- **Mapa principal**: camino sinuoso de nodos conectados por una línea dibujada a mano en Canvas, no una lista vertical de botones.

7. Pruebas

135 tests totales: **134 JVM** (JUnit4, algunos con Robolectric + Room en memoria) + **1 instrumentado** (Compose UI test de humo). Ver desglose completo en `BUILD_REPORT.md`.

8. Compilación

```
./gradlew clean
./gradlew testDebugUnitTest
./gradlew lintDebug
./gradlew assembleDebug
```

Importante: este proyecto se generó en un entorno sin Android SDK, sin JDK+Kotlin compiler y sin acceso de red (verificado, ver `BUILD_REPORT.md`). Ninguno de estos comandos se ejecutó realmente. El repositorio incluye `.github/workflows/build.yml` para ejecutar la compilación y los tests reales vía GitHub Actions al hacer push.

9. Extender el contenido

Para añadir nuevos desafíos semilla: editar `tools/generate_seed_content.py` (nunca `SeedContent.kt` a mano — perdería la garantía de auto-verificación) y volver a ejecutar `python3 tools/generate_seed_content.py`.